

JAVA QB UNIT – 1 (2 marks)

1. Define JDBC.

1. JDBC stands for **Java Database Connectivity**. It is a standard Java API used to connect Java programs with databases.
2. It provides classes and interfaces to send SQL queries to the database. It also helps in receiving results from the database.
3. JDBC acts as a bridge between Java applications and relational databases. It allows communication with databases like MySQL and Oracle.
4. JDBC is included in Java and available in the java.sql package. It makes database programming easier in Java.

2. What is the role of JDBC in Java applications?

1. JDBC helps Java applications connect to databases. Without JDBC, Java cannot directly talk to a database.
2. It allows sending SQL commands like INSERT, UPDATE, DELETE, and SELECT. It also receives results in Java form.
3. JDBC ensures database-independent programming. The same Java code can work with different databases.
4. It helps manage connections, execute queries, and process results. It is essential for real-world data storage applications.

3. Explain the reason behind JDBC.

1. Different database vendors provide their own APIs. These APIs are not standard and are vendor-specific.
2. If we use vendor-specific APIs, the code must be rewritten when changing the database. This increases development time.
3. JDBC provides a common and standard API. It allows Java programs to work with different databases easily.
4. JDBC improves portability and reduces dependency on a single database vendor. It makes database access uniform and simple.

4. What is the purpose of DriverManager class?

1. The DriverManager class manages all the JDBC drivers loaded in the system. It acts as a connection manager.
2. It establishes a connection between Java application and database. It uses database URL, username, and password.
3. It selects the appropriate driver for the database automatically. This makes database connection easier.
4. It returns a Connection object. This object is used to execute SQL statements.

5. Which package contains JDBC classes and interfaces?

1. JDBC classes and interfaces are available in the java.sql package. This package is part of Java API.
2. It contains important interfaces like Connection, Statement, ResultSet, and PreparedStatement. These help in database operations.
3. It also contains classes like DriverManager and SQLException. These are used for managing connections and handling errors.
4. The java.sql package provides all necessary tools for database programming in Java. It is required to use JDBC.

6. What is the difference between executeQuery() and executeUpdate()?

1. executeQuery() is used for SELECT statements. It returns a ResultSet object containing data.
2. executeUpdate() is used for INSERT, UPDATE, DELETE, CREATE, and DROP statements. It returns an integer value.
3. The integer returned by executeUpdate() shows the number of rows affected. executeQuery() does not return row count.
4. Use executeQuery() when retrieving data. Use executeUpdate() when modifying data in the database.

7. List the four types of JDBC drivers.

1. Type 1 – JDBC-ODBC Bridge Driver. It converts JDBC calls into ODBC calls.
2. Type 2 – Native-API Driver. It uses database-specific native libraries.
3. Type 3 – Network Protocol Driver. It uses middleware between client and database.
4. Type 4 – Thin Driver. It is a pure Java driver and most widely used.

8. Differentiate between Statement and PreparedStatement.

1. Statement is used for simple SQL queries without parameters. PreparedStatement is used for parameterized queries.
2. In Statement, SQL is compiled every time it is executed. In PreparedStatement, SQL is precompiled for better performance.
3. Statement is less secure and can cause SQL injection. PreparedStatement is more secure because it uses placeholders (?).
4. Statement is suitable for simple queries. PreparedStatement is better for repeated execution and dynamic values.

9. What is a CallableStatement used for?

1. CallableStatement is used to call stored procedures in the database. It extends the PreparedStatement interface.
2. It allows passing input parameters using question marks (?). It can also receive output parameters.
3. It is useful when database logic is written as procedures. It helps execute complex database operations.
4. It uses methods like setXXX() and getXXX() for parameters. It executes procedures using the execute() method.

10. Name the two types of ResultSet.

1. Scrollable ResultSet. It allows moving forward and backward in the records.
2. Updatable ResultSet. It allows updating the data directly in the ResultSet.
3. Scrollable ResultSet supports cursor methods like next(), previous(), first(), and last(). It improves navigation.
4. Updatable ResultSet allows modifying data and reflecting changes in the database. It must be created explicitly.

11. Write any two database operations supported by JDBC.

1. INSERT operation. It is used to add new records into a database table.
2. SELECT operation. It is used to retrieve data from the database.
3. UPDATE operation. It is used to modify existing records in a table.
4. DELETE operation. It is used to remove records from a table.

12. Mention one advantage of using PreparedStatement over Statement.

1. PreparedStatement is more secure than Statement. It prevents SQL injection attacks.
2. It uses placeholders (?) for parameters. This makes query execution safer.
3. It is precompiled by the database. This improves performance for repeated queries.
4. It is better for dynamic values provided by the user.

13. Explain Class.forName() statement.

1. Class.forName() is used to load the JDBC driver class. It loads the driver into memory.
2. It is required before establishing a database connection. Without loading the driver, connection cannot be made.
3. Example: Class.forName("com.mysql.cj.jdbc.Driver");. This loads MySQL driver.
4. If the driver class is not found, it throws ClassNotFoundException.

14. Write the statement for creating a connection with database.

1. Connection is created using DriverManager class. It connects Java program to the database.
2. Syntax:
Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/college",
"root","password");
3. It requires JDBC URL, username, and password. These details identify the database.
4. It returns a Connection object. This object is used to execute SQL statements.

15. What is the purpose of next() method of ResultSet interface? What happens if you do not call next() before reading ResultSet data?

1. The next() method moves the cursor to the next record in the ResultSet. It returns true if a record exists.
2. The cursor initially points before the first record. So we must call next() to access data.
3. If next() is not called, the program cannot read data. It may throw an exception.
4. next() is usually used inside a while loop. It helps in reading all records one by one.

16. What is the use of placeholders (?) in PreparedStatement?

1. Placeholders (?) are used to represent values in a SQL query. These values are given later using setXXX() methods.
 2. They allow dynamic input from the user. This makes the query flexible and reusable.
 3. Placeholders improve security. They help prevent SQL injection attacks.
 4. They also improve performance because the query is precompiled once. The same query can be executed many times with different values.
-

17. Differentiate between executeQuery(), executeUpdate(), and execute().

1. executeQuery() is used for SELECT statements. It returns a ResultSet object containing data.
2. executeUpdate() is used for INSERT, UPDATE, DELETE, CREATE, and DROP statements. It returns an integer showing number of rows affected.
3. execute() can be used for any type of SQL statement. It returns a boolean value.
4. execute() returns true if the result is a ResultSet. It returns false if the result is an update count.

(5 marks)

1. Explain the architecture of JDBC and its main components.

1. JDBC architecture is used to connect a Java application with a database. It acts as a bridge between Java program and database system. It allows sending SQL queries and receiving results.
2. The first main component is **JDBC API**. It contains classes and interfaces like Connection, Statement, and ResultSet. These are available in the java.sql package.
3. The second component is **DriverManager**. It manages database drivers and creates a connection between Java and the database. It selects the correct driver based on the database URL.
4. The third component is the **JDBC Driver**. It converts JDBC calls into database-specific commands. Different databases require different drivers.
5. The last component is the **Database**. It stores data and executes SQL queries sent from the Java application. The result is returned back through the driver to the Java program.

2. Explain two-tier and three-tier JDBC architecture with advantages and disadvantages.

Two-Tier Architecture

1. In two-tier architecture, the Java application directly connects to the database. The client contains user interface, business logic, and JDBC code.
2. The JDBC driver directly communicates with the database server. SQL queries are sent directly from the client to the database.

Advantages:

3. It is simple and easy to design. It is suitable for small desktop applications.
4. Communication is faster because there is direct connection. It is good for small number of users.

Disadvantages:

5. It has poor scalability for large systems. It is difficult to handle many users.
 6. Database credentials are exposed at client side. This reduces security.
-

Three-Tier Architecture

1. In three-tier architecture, the client does not directly connect to the database. There is a middle layer (application server) between client and database.
2. The middle layer handles business logic, security, and JDBC operations. The database layer stores and manages data.

Advantages:

3. It provides better security because database access is controlled by middle layer. It is suitable for web and enterprise applications.
4. It supports large number of users. It is highly scalable and easy to maintain.

Disadvantages:

5. It is more complex to design. Development cost is higher.
6. It requires an application server. System setup is more complicated.

3. Describe the steps involved in creating a JDBC application.

1. The first step is loading the JDBC driver. This is done using `Class.forName()` method. It loads the driver class into memory.
2. The second step is establishing a connection. This is done using `DriverManager.getConnection()` with database URL, username, and password. It returns a `Connection` object.
3. The third step is creating a `Statement` or `PreparedStatement`. This object is used to send SQL queries to the database.
4. The fourth step is executing SQL query. Methods like `executeQuery()` or `executeUpdate()` are used depending on the type of SQL statement.

5. The fifth step is processing the ResultSet. The next() method is used to move through records and retrieve data using getXXX() methods.
6. The final step is closing the connection. ResultSet, Statement, and Connection objects must be closed to free resources.

4. Compare the four types of JDBC drivers with suitable examples.

1. **Type 1 – JDBC-ODBC Bridge Driver.** It converts JDBC calls into ODBC calls. It is slow and platform dependent. It was removed after Java 8.
2. **Type 2 – Native API Driver.** It uses database-specific native libraries. It provides better performance than Type 1 but is not fully portable.
3. **Type 3 – Network Protocol Driver.** It uses middleware between client and database. It is fully written in Java and platform independent. It is slower due to network communication.
4. **Type 4 – Thin Driver.** It is a pure Java driver and directly communicates with the database. It provides best performance and is most widely used.
5. Example of Type 4 driver is MySQL driver: com.mysql.cj.jdbc.Driver. It does not require any native libraries. Among all drivers, Type 4 is most preferred in modern applications. It is fast, portable, and secure.

5. Explain different types of JDBC drivers with their advantages and disadvantages.

1. Type 1 – JDBC-ODBC Bridge Driver.

This driver converts JDBC calls into ODBC calls. It is platform dependent and was removed after Java 8.

Advantage: It is easy to use for learning purposes.

Disadvantage: It is slow and less secure. It requires ODBC driver installation on client machine.

2. Type 2 – Native API Driver.

This driver uses database-specific native libraries. It is partially written in Java.

Advantage: It gives better performance than Type 1 driver.

Disadvantage: It is not portable and requires vendor libraries to be installed.

3. **Type 3 – Network Protocol Driver.**

This driver uses middleware between client and database. It is fully written in Java.

Advantage: It is platform independent and database independent.

Disadvantage: It is slower due to network overhead and middleware maintenance cost.

4. **Type 4 – Thin Driver.**

This driver directly converts JDBC calls into database protocol. It is fully written in Java.

Advantage: It provides best performance and is platform independent.

Disadvantage: It is database dependent because it is made for a specific database.

6. **Discuss Connection interface with its methods.**

1. The Connection interface is part of the java.sql package. It represents a session between Java application and database.
 2. It is created using DriverManager's getConnection() method. It allows communication with the database.
 3. The method createStatement() is used to create a Statement object. This object sends SQL queries to the database.
 4. The method prepareStatement(String sql) is used to create a PreparedStatement object. It is used for parameterized queries.
 5. The method prepareCall(String sql) is used to create a CallableStatement. It is used for calling stored procedures.
 6. Methods like commit() and rollback() are used for transaction management. They help control database changes.
 7. The close() method is used to close the connection. It releases database resources.
-

7. What do you understand by Statement? Explain different types of statement in JDBC.

1. A Statement is an interface used to execute SQL queries in JDBC. It sends SQL commands from Java program to the database.
 2. A Statement object is created using `connection.createStatement()`. It is mainly used for simple and static SQL queries.
 3. **Statement:**
It is used for normal SQL queries without parameters. It is suitable for simple queries like SELECT or DELETE.
 4. **PreparedStatement:**
It is used for parameterized queries. It improves performance because the query is precompiled.
 5. **CallableStatement:**
It is used to call stored procedures in the database. It supports both input and output parameters.
 6. These three types of statements help execute different kinds of database operations efficiently.
-

8. Explain PreparedStatement with appropriate code snippet.

1. PreparedStatement is used to execute parameterized SQL queries. It is a sub-interface of Statement.
2. It allows using placeholders (?) for dynamic values. These values are set using `setXXX()` methods.
3. It improves performance because the SQL query is precompiled. It also prevents SQL injection.
4. It is created using `prepareStatement()` method of Connection interface.

5. Example Code:

```
import java.sql.*;

class InsertData {
    public static void main(String args[]) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college", "root", "");

            PreparedStatement pstmt = con.prepareStatement(
                "INSERT INTO student VALUES (?, ?)");

            pstmt.setInt(1, 101);
            pstmt.setString(2, "Riya");

            int i = pstmt.executeUpdate();
            System.out.println(i + " record inserted");

            con.close();
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

6. In this example, ? are placeholders. The values are set using setInt() and setString() methods before execution.

9. Compare Statement and PreparedStatement with proper use cases.

1. **Statement** is used for simple SQL queries without parameters. It is suitable when the query is fixed and executed only once.
 2. In Statement, the SQL query is compiled every time it is executed. This makes it slower for repeated execution.
 3. **Use Case of Statement:**
It is used in small programs where query does not change. Example:
`SELECT * FROM student;`
 4. **PreparedStatement** is used for parameterized queries. It allows placeholders (?) for dynamic values.
 5. It is precompiled by the database. This improves performance when the same query runs multiple times.
 6. **Use Case of PreparedStatement:**
It is used when user input is involved, such as login forms or inserting student details. It prevents SQL injection and improves security.
-

10. Discuss the differences between Statement, PreparedStatement, and CallableStatement with use cases.

1. **Statement** is used for simple and static SQL queries. It does not support parameters.
Use Case: Displaying all records from a table.
2. **PreparedStatement** is used for parameterized SQL queries. It uses placeholders (?) and setXXX() methods.
Use Case: Inserting user details or searching records based on user input.
3. PreparedStatement is faster than Statement for repeated execution. It also provides better security.
4. **CallableStatement** is used to call stored procedures in the database. It can handle both input and output parameters.
5. CallableStatement is useful when database logic is already written as procedures. It is commonly used in enterprise applications.

6. In summary, Statement is for simple queries, PreparedStatement is for secure dynamic queries, and CallableStatement is for stored procedures.
-

11. Explain Scrollable and Updatable ResultSet with examples.

1. A default ResultSet moves only forward and is read-only. To use special features, we must create Scrollable or Updatable ResultSet explicitly.
2. **Scrollable ResultSet** allows moving forward and backward in the result set. Methods like next(), previous(), first(), and last() are used.

Example:

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_READ_ONLY);
```

This allows moving in both directions but does not allow updating data.

3. **Updatable ResultSet** allows modifying data directly in the result set. Changes can be reflected in the database.

Example:

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);
```

This allows updating records.

4. Scrollable ResultSet improves navigation. Updatable ResultSet allows updating records without writing separate SQL queries.

12. Explain ResultSet interface with its methods.

1. ResultSet interface represents the data returned by a SELECT query. It works like a table of rows and columns.
2. The cursor initially points before the first row. We must call next() to move to the first record.
3. Important getter methods include getInt(), getString(), getFloat(), and getDate(). These methods retrieve column values.
4. Navigation methods include next(), previous(), first(), last(), and absolute(int row). These help move the cursor.
5. ResultSet can be forward-only, scrollable, or updatable. The type depends on how it is created.
6. After processing data, we must call rs.close(). This releases database resources.

13. Explain different types of ResultSet with appropriate example.

1. A ResultSet stores the data returned by a SELECT query. By default, it is forward-only and read-only.

2. Forward-Only ResultSet

The cursor moves only in forward direction using next(). Data cannot be modified.

Example:

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

3. Scrollable ResultSet

The cursor can move forward and backward. Methods like previous(), first(), and last() are used.

Example:

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_READ_ONLY);
```

4. Updatable ResultSet

It allows updating records directly in the ResultSet. Changes are reflected in the database.

Example:

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATABLE);
```

5. Scrollable ResultSet improves navigation. Updatable ResultSet allows modifying records without separate update query.

14. Write a JDBC program to delete a record from a table.

```
import java.sql.*;
```

```
class DeleteRecord {  
    public static void main(String args[]) {  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/college",  
                "root",  
                "password");  
  
            Statement stmt = con.createStatement();  
  
            int rows = stmt.executeUpdate(  
                "DELETE FROM student WHERE id = 101");  
  
            System.out.println(rows + " record deleted");  
  
            stmt.close();  
            con.close();  
  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
    }  
}
```


Explanation:

1. Driver is loaded using Class.forName().
2. Connection is created using DriverManager.
3. executeUpdate() is used for DELETE query.
4. It returns number of rows affected.

15. Write a JDBC program to update student marks using PreparedStatement.

```
import java.sql.*;

class UpdateMarks {
    public static void main(String args[]) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/college",
                "root",
                "password");

            PreparedStatement pstmt = con.prepareStatement(
                "UPDATE student SET marks = ? WHERE id = ?");

            pstmt.setInt(1, 85); // new marks
            pstmt.setInt(2, 101); // student id

            int rows = pstmt.executeUpdate();

            System.out.println(rows + " record updated");

            pstmt.close();
            con.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

Explanation:

1. PreparedStatement uses placeholders (?).
2. Values are set using setInt() method.
3. executeUpdate() updates the record.

16. Explain Scrollable ResultSet with cursor methods and example.

1. A Scrollable ResultSet allows moving the cursor both forward and backward. It is created using special constants.
2. It is created using:
ResultSet.TYPE_SCROLL_INSENSITIVE or
ResultSet.TYPE_SCROLL_SENSITIVE.
3. Common cursor methods are:
 - next() → move to next record
 - previous() → move to previous record
 - first() → move to first record
 - last() → move to last record
 - absolute(int row) → move to specific row

4. Example:

```
Statement stmt = con.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_READ_ONLY);
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

```
rs.last();
```

```
System.out.println("Last Student: " + rs.getString("name"));
```

```
rs.first();
```

```
System.out.println("First Student: " + rs.getString("name"));
```

5. Scrollable ResultSet improves data navigation. It is useful when user needs to move between records freely.

